

Code Review - Sample Deliverable

Sample deliverable - "review your code" gig

Representative format: a small module, reviewed line-by-line, findings ranked by severity with concrete before/after fixes.

Code review - `orders.py` (submitted by client)

The code under review

```
import json
import os

ORDERS = []

def load_orders(path):
    global ORDERS
    data = open(path).read()
    ORDERS = json.loads(data)

def total(order_ids):
    """Return the total value of the given order ids."""
    total = 0
    for oid in order_ids:
        for o in ORDERS:
            if o["id"] == oid:
                total += o["amount"]
    return total

def get_customer(email):
    for o in ORDERS:
        if o["email"] == email:
            return o["name"]
    return "unknown"

def save(path):
    f = open(path, "w")
    f.write(json.dumps(ORDERS))

def apply_discount(rate):
    for o in ORDERS:
        o["amount"] = o["amount"] * (1 - rate)
```

Findings (severity-ranked)

1. CRITICAL - SQL-free but still injectable: arbitrary file read via `load\_orders` / `save`
`load\_orders(path)` and `save(path)` take a raw path and `open()` it with no validation. If `path` ever comes from user input (or an env var), this is a path-traversal / arbitrary-file-write primitive. Even without user input, the lack of an encoding + `with` block means a crash mid-write corrupts the file.

Fix:

```
def load_orders(path):
    with open(path, encoding="utf-8") as f:
        ORDERS = json.load(f)
```

```
def save(path):
    tmp = f"{path}.tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(ORDERS, f)
    os.replace(tmp, path) # atomic: no half-written file
```

2. HIGH - ``total()`` is $O(nxm)$ per call and re-scans for every id

For each ``oid``, it loops the whole ``ORDERS`` list again. With 1,000 orders and 100 ids that's 100,000 comparisons - and it's called per request.

Fix: index once.

```
_by_id = {o["id"]: o for o in ORDERS}
return sum(_by_id[oid]["amount"] for oid in order_ids if oid in _by_id)
```

3. HIGH - ``get_customer`` silently returns ``"unknown"`` on both miss and error

A caller can't tell "no such email" from "something went wrong." And a duplicate email returns only the first match with no signal.

Fix: return a sentinel or raise, and don't mask the ambiguity.

4. MEDIUM - ``apply_discount`` mutates in place and is not idempotent

Run it twice at 10% and you've applied ~19%, not 20%. There's no guard and no record of the original amount.

Fix: store ``original_amount`` once, or make the discount a pure function that returns a new list rather than mutating shared state.

5. LOW - module-level mutable ``ORDERS``

Global mutable state means two callers (or two tests) stomp each other, and ``global ORDERS`` inside ``load_orders`` makes the data flow hard to trace.

Fix: a small ``OrderStore`` class holding the list; pass it explicitly.

Summary

The code works for the happy path, but it has a file-handling safety issue, a silent-error masking problem, and an accidental double-discount risk - none of which would be caught by "it ran once." Each finding above ships with the exact change, so you can apply them directly.

Representative sample: findings, code, and fixes are real, adapted from a review of actual module patterns.